

Suport de curs:
Fiabilitate și Diagnoză,
Fiabilitate Software,
Sisteme Tolerante la Defecte

dr. ing. Camil Băncioiu

May 12, 2025

Chapter 1

Introducere

Acest curs explorează ideile și practicile ingineresti care vizează fiabilitatea sistemelor. Domeniul fiabilității este larg și greu de acoperit într-un singur curs, dar totodată e un domeniu foarte important. Acest curs încearcă să familiarizeze studenții cu abordările, atitudinile și rigorile necesare acestui domeniu și încearcă în permanență să ofere exemple cât se poate de concrete și reale. Din păcate, asta duce la compromiterea aprofundării.

Autorul cursului speră că varietatea subiectelor prezentate va oferi puncte de pornire pentru discuții viitoare despre fiabilitate în multe din echipele în care actualii studenți vor fi implicați.

1.1 Fiabilitate

O definiție naivă a fiabilității spune că un sistem este fiabil dacă, după dezvoltare și producție, rămâne în parametrii săi de funcționare normală îndeajuns de mult timp.

1.1.1 Îndeajuns de mult timp - cât?

Depinde de intențiile cu care a fost proiectat sistemul și constrângerile cu care inginerii s-au confruntat în timpul dezvoltării. O bicicletă este considerată fiabilă dacă în primii câțiva ani de folosire are nevoie doar de mentenanță minimală, și posesorul o poate folosi mulți ani. Zece ani este o durată de folosire continuă acceptabilă pentru o bicicletă.

Dar zece ani ar fi insuficienți pentru o centrală nucleară, în construcția căreia au fost investite mult efort și multe fonduri. Și mai ales pentru că durează între 5 și 10 ani doar ca centrala să fie construită, înainte de a fi pusă în funcțiune. De aceea, centralele nucleare construite mai recent sunt proiectate să funcționeze fiabil în jur de o jumătate de secol (deși este de dorit să poată fi folosite și mai mult de atât).

Un secol de funcționare fiabilă este însă prea puțin pentru Svalbard Global Seed Vault (SGSV), depozitul de rezervă din Norvegia deschis în 2008 în care sunt păstrate semințe ale speciilor de plante importante în agricultură. Misiunea SGSV este de a oferi o rezervă în cazul pierderii accidentale a accesului la bănci genetice sau de semințe din lume. Această situație a fost deja întâmpinată în 2015, când Centrul Internațional pentru Cercetare Agricolă în Regiunile Aride (International Center for Agricultural Research in the Dry Areas, ICARDA) a fost nevoit să retragă din SGSV o parte din semințe în 2015 și 2017 pentru a le cultiva și multiplica, în urma pierderii accesului la facilitățile proprii din Tel Hadya, Siria, datorată războiului civil. Din fericire, bombardamentele nu au afectat facilitățile atât de importante, dar războiul a împiedicat accesul la semințele stocate acolo și distribuția lor. Aceste două retrageri au fost singurele efectuate de când SGSV a fost deschis. Dar nu au trecut nici măcar 10 ani de la deschidere și deja s-a dovedit nevoia acestui depozit excepțional. Oricum, SGSV este proiectat să păstreze semințe pentru mai multe secole, cu potențial pentru câteva milenii.

Câteva milenii pare o perioadă lungă, dar este mult prea puțin pentru depozitarea combustibilului nuclear epuizat, provenit din multele centrale nucleare cu fisiune din lume. Combustibilul folosit în reactoare (dioxid de uraniu sau amestecuri de uraniu cu plutoniu) devine tot mai radioactiv de-a lungul perioadei de folosire, ceea ce îi scade utilitatea în reactor. După doar câțiva ani, materialul trebuie înlocuit, ducând astfel la nevoia de depozitare a combustibilului nuclear considerat în acest fel epuizat. Perioada de înjumătățire a acestor

materiale este de ordinul zecilor de mii de ani, sau în unele cazuri sute de mii de ani. Și trecerea unei perioade de înjumătățire face doar atât: înjumătățește cantitatea de material original. Trebuie să treacă mai multe perioade pentru ca materialul să poată fi considerat acceptabil (în niciun caz nu va fi ”sigur”).

Deocamdată, depozitarea combustibilului nuclear epuizat nu este o problemă rezolvată, deoarece depozitele necesită mentenanță și supraveghere continuă pe o durată extrem de lungă de timp. Materialul epuizat este intens radioactiv, ceea ce degradează destul de ușor spațiul în care este păstrat. Sistemele electronice care supraveghează depozitarea au marjă de eroare foarte mică, și trebuie să rămână în acea marjă de eroare pentru mult timp, chiar dacă nu se așteaptă nimeni ca ele să reziste peste 10 000 de ani.

1.2 Fiabilitate și simplitate

În mod intuitiv, știm că un sistem simplu ar trebui să aibă mai puține probleme, deci va fi mai fiabil, cu durată de folosire mai lungă. Dar un sistem complex, cu multe componente care interacționează, va avea multe oportunități pentru apariția problemelor.

Asta nu înseamnă că un sistem simplu este în mod implicit un sistem fiabil, sau că un sistem complex nu poate fi făcut să fie fiabil. Cu îndeajuns de mult efort, orice sistem poate fi făcut fiabil.

Dar dacă urmărim simplitatea ca principiu general, reducem mult din sursele de probleme, cum ar fi eroarea umană la proiectare și construcție, eroarea umană la folosire, interacțiunea neprevăzută a unor componente izolate, lipsa de interacțiune între componente care depind una de cealaltă sau multe altele.

Cuvântul ”componentă” este folosit aici în mod general, însemnând și componentă fizică a unui ansamblu (componentă hardware), dar și un modul de cod într-un program (componentă software). Fiecare componentă individuală trebuie să fie fiabilă. Ansamblurile de componente trebuie să fie fiabile, la rândul lor. Fiabilitatea proiectul integral trebuie luată în considerare, cu toate sistemele și subsistemele lui.

Acest principiu este la fel de valabil pentru proiecte software, cum ar fi o aplicație de mesagerie, joc video sau sistem de operare, precum și pentru proiecte hardware, cum ar fi un sistem de coordonare a semafoarelor rutiere, o rețea de turnuri de telecomunicație sau o centrală nucleară.

Indiferent de tipul de sistem, rămâne valabil acest principiu:

Simplicity is prerequisite for reliability.

— Edsger Dijkstra, EWD498, 1975

<https://www.cs.virginia.edu/~evans/cs655/readings/ewd498.html>

Dar opusul simplității, complexitatea, poate fi rareori evitată. De obicei ne confruntăm cu sisteme complexe, cu multe componente care interacționează.

1.2.1 Simplu sau ușor. Complex sau dificil.

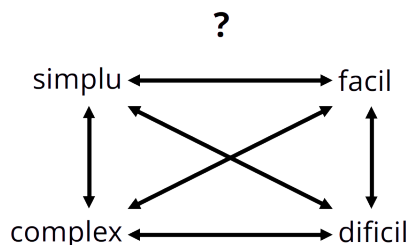


Figure 1.1: Relații între termeni

A se observa relațiile dintre ”simplu” și ”complex”, dintre ”facil” și ”dificil”.

În această diagramă, avem doar două perechi de antonime: *simplu vs. complex* și *facil vs. dificil*. Asta înseamnă că un sistem poate fi simplu și dificil, sau facil și complex, dar nu poate fi simplu și complex deodată. Dar dacă urmărim simplitatea ca principiu general, complexitatea poate reieși fără dificultatea pe care i-o atribuim în mod intuitiv.

Când discutăm despre fiabilitate, ținem cont de faptul că un sistem simplu oferă mai puține oportunități pentru defecte, cum este menționat mai sus, dar ceea ce oferă cu adevărat fiabilitate este favorizarea **testării**. Despre testare va fi discutat în Capitolul 2

1.3 Dezvoltare și producție

Există o oarecare confuzie legată de termenul ”în producție” când vine vorba de software, pentru că deseori este folosit încălcând paralela cu producția de hardware. Nu există un consens formal, dar în limbajul uzual, software ”în producție” se referă la software care a fost livrat, instalat la client sau publicat – în afara condițiilor controlate, de dezvoltare. În cazul hardware, producția se întâmplă încă în fabrică sau atelier, în condițiile controlate ale producătorului.

Însă acestei viziuni îi lipsește o definiție concretă pentru *produs* software, în ciuda faptului că auzim ”producție” în multe contexte. O definiție utilă pentru ”produs” software ar fi comportamentul unei instanțe individuale a software-ului pe un singur dispozitiv (telefon mobil, calculator personal, microcontroller etc.), produsă de dispozitiv la cererea utilizatorului, prin rularea unei copii a unui executabil instalat pe dispozitivul individual; executabilul a fost, desigur, dezvoltat de ingineri scriind cod-sursă.

A se reține că această interpretare nu este neapărat răspândită, dar autorul acestui document va insista că este mai utilă și mai aproape de realitatea hardware decât cea uzuală, întâlnită în domeniu. Ar fi mai utilă din cauză că oferă o analogie mai bună cu dezvoltarea și producția hardware, explicând mai bine analogiile și paralelele între hardware și software.

Dar acest document va încerca să rămână la interpretarea uzuală, răspândită în domeniul industriei de software comercial: ”produsul” software este rezultatul dezvoltării software, în cel mai general înțeles.

1.4 Software

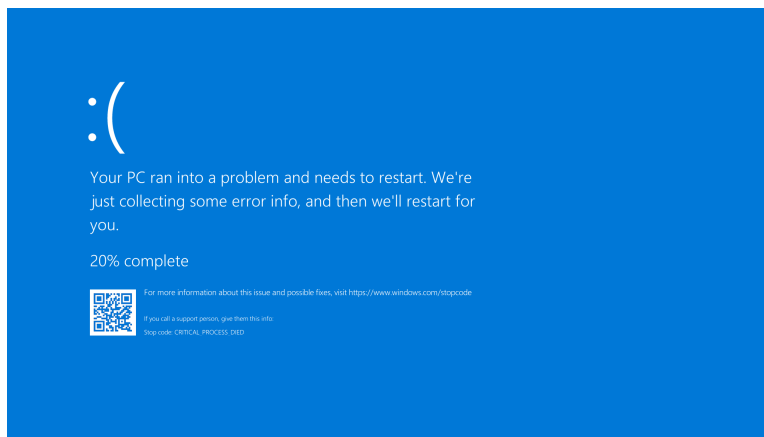


Figure 1.2: ”Blue Screen of Death”, Windows 10

În multe din discuții, acest document va urmări fiabilitatea sistemelor software, dar vor fi atinse și aspecte de fiabilitate hardware.

Împărțim de regulă sistemele software în următoarele categorii:

- **biblioteci:** acestea sunt sisteme software care oferă funcționalitate altor sisteme; bibliotecile sunt *apelate* de către aplicația care le folosește, fiind oarecum pasive, dar extrem de eficace în reducerea eforturilor de implementare (prin reutilizarea codului) și, de multe ori, oferă fiabilitate prin expertiza dezvoltatorilor bibliotecii (de exemplu, bibliotecile criptografice);
- **framework-uri:** acestea sunt sisteme software folosite pentru a construi alte sisteme; cum sugerează numele, un framework urmărește analogia unei schele; framework-ul oferă o structură prestabilită, dar configurabilă, și oferă dezvoltatorilor posibilitatea de a plasa funcționalitate personalizată într-un cadru; avantajul este reutilizarea structurii de aplicație și degrevarea dezvoltatorilor de efortul de a implementa acest cadru, rămânând doar efortul de implementare a funcționalității dorite;

- **firmware:** acestea sunt sisteme software dedicate unor componente fizice, strâns legate de resursele componentei; un firmware rulează, în mod normal, doar pe componenta căreia îi este destinat (sau într-un simulator de componentă);
- **aplicații:** acestea sunt sisteme software care sunt destinate interacțiunii cu oamenii; includem aici aplicațiile desktop (sau laptop), aplicațiile pentru dispozitive mobile, și aplicațiile pentru console; jocurile sunt incluse tot la aplicații.

Un caz special de aplicații sunt aplicațiile pentru servere, aplicațiile "enterprise" și aplicațiile științifice. Pentru discuțiile din documentul curent, le considerăm tot aplicații, împreună cu cele enumerate mai sus.

De multe ori, în locul denumirii software, sistem informatic sau securitate informatică, auzim termenii sistem cibernetic sau securitate cibernetică. Este important de reținut că termenul cibernetic nu este folosit corect aici, chiar dacă este extrem de răspândit. Foarte mulți folositori nu vor ști să dea o definiție corectă pentru "cibernetic".

Conform definiției reale, cibernetica este o ramură a teoriei sistemelor, și studiază sistemele care conțin bucle de cauzalitate între intrările și ieșirile lor. Cu alte cuvinte, studiază sistemele care includ propriile lor reacții în intrări. Altfel spus, putem vedea aceste sisteme ca fiind autoguvernante, lucru confirmat de etimologia termenului (limba greacă κυβερνητικός, citit "kubernētikos", rom. "cârmaci"; compară cu "guvernator")

Cibernetica nu a avut la origine conotații de "tehnologic", "interconectat", "informatic" sau "digitalizat". Aceste conotații au venit din literatura science-fiction, cu precădere curentul cyberpunk.

Autorul acestui document recomandă folosirea termenului informatic în locul lui cibernetic.

Chapter 2

Testarea

Introducerea (Capitolul 1) a accentuat urmărirea simplității în cât mai multe cazuri. Unul din cele mai importante motive pentru a prioritiza simplitatea este de a favoriza **testarea**, mai ales **testarea automată**.

Ne dorim testare cât mai multă, în toate etapele dezvoltării și producției, din cauză că testarea este modul concret prin care validăm faptul că rezultatul muncii noastre respectă așteptările noastre. În această discuție, ”rezultatul muncii noastre” este un *sistem* de orice fel, fie software, fie hardware.

Testarea are loc în timpul dezvoltării, sau cel mai târziu în timpul producției, și se încheie înainte ca sistemul să fie livrat sau publicat. După livrare sau publicare, intervin alte tipuri de validare ale sistemului, un exemplu fiind *invariantii*, despre care se va discuta în Capitolul 4.

Prin *testare*, în cel mai general sens, înțelegem următorul procedeu:

1. *alegerea unei părți* pe care o supunem testării (sau întregul sistem),
2. *delimitarea părții* de restul sistemului (unde e cazul),
3. *definirea așteptărilor* noastre (**scenariile** și **rezultatele** așteptate),
4. *parcurea scenariilor* de testare,
5. *culegerea rezultatelor* în urma parcurgerii scenariilor,
6. *evaluarea rezultatelor*.

În ultima etapă, *evaluarea rezultatelor*, vom ști dacă partea supusă testării se conformează așteptărilor noastre sau, dacă nu, ce anume mai are nevoie încă de lucru.

Este important de reținut că testarea nu e

- nici *diagnoză*, dar unele teste pot fi într-adevăr *teste de diagnoză*, care pot confirma prezența unei anume probleme suspectate,
- nici *depanare*, dar testarea este cea care validează înlăturarea cu succes a ”penei”.

În schimb, *testarea este un prag care trebuie trecut* cu succes, și de care nu se trece decât dacă sunt îndeplinite criteriile cerute de scenariile de testare. Desigur, testarea este de foarte multe ori o parte integrală a proceselor de diagnoză și de depanare.

În funcție de circumstanțele în care sistemul nostru e dezvoltat și produs, putem opta pentru *testare manuală* sau *testare automată*, ori o combinație între cele două.

Testarea manuală implică parcurgerea scenariilor de testare, a culegerii și evaluării rezultatelor fără a folosi procese automatizate. Testarea manuală este consumatoare de timp, vulnerabilă la ”factorul uman” (neatenție, greșeli operative sau de manipulare, necunoaștere, lipsă de motivație etc.) și dificil de extins.

Testarea automată implică parcurgerea scenariilor de testare, a culegerii și evaluării rezultatelor folosind procese automatizate. Testarea automată este foarte rapidă (cu unele excepții), repetabilă și ușor de extins. Din aceste motive, procesele de testare automată sunt efectuate de mai multe ori și în multe din fazele de dezvoltare / producție.

Chiar dacă testarea automată necesită mai mult efort de pregătire inițială, efortul se amortizează foarte rapid. Din acest motiv, se recurge la testare automată în aproape toate cazurile. Mai mult, testarea automată este de multe ori dominantă chiar și atunci când se adoptă o practică hibridă de testare manuală cu automată.

Testarea manuală poate fi totuși preferabilă în cazul producției pe scară foarte mică sau unicat, sau în cazul prototipurilor simple.

2.1 Automatizarea testării

2.1.1 Hardware

În cazul **sistemelor hardware**, numărul de articole produse tinde să fie cel care dictează nivelul de automatizare al testării. Acest fapt este o consecință a automatizării producției, ceea ce înseamnă că testarea manuală devine insuficientă. Iar în cazul sistemelor hardware complexe produse în masă, testarea manuală se poate rezuma la asigurarea unor inspecții efectuate la finalul unor etape, înainte de asamblarea finală sau înainte de sigilarea produsului.

Dar înainte de producției (și a testării din producție), sistemele hardware sunt, desigur, dezvoltate în laboratoare și ateliere, unde lipsește automatizarea unei linii de producție. Totodată, în faza de dezvoltare se testează doar unul sau mai multe prototipuri, în repetate rânduri.

Atunci când se discută și se pregătesc scenariile de test și criteriile specifice pentru o anumită componentă sau ansamblu hardware, le numim simplu *dispozitivul testat* (eng. Device Under Test, DUT), *unitatea testată* (eng. Unit Under Test) sau *sistemul testat* (eng. System Under Test), în funcție de componentă sau ansamblu.

Un caz interesant este al sistemelor hardware care includ componente electronice, de obicei sub forma PCB (eng. Printed Circuit Board; circuit imprimat). O placă de circuit imprimat este prevăzută cu *puncte de test* (eng. test points) cablate la componentele montate anterior pe placă. Înainte ca placa să fie instalată, ea este supusă unei testări automate folosind un *dispozitiv de testare* (eng. test fixture), care este prevăzut cu contacte special amplasate pentru a atinge punctele de test ale plăcii. În acest fel, dispozitivul de testare oferă posibilitatea de a testa individual fiecare placă, înainte de a fi instalată în ansamblul-părinte.

Un dispozitiv de testare este în marea majoritate a cazurilor fie programabil în sine, fie conectabil la un calculator de comandă. În orice caz, menirea dispozitivului de testare este de a rula scenarii de test specific create pentru placa în cauză.

Câteva exemple de verificări care pot fi efectuate în dispozitivul de test:

- se verifică conectivitatea circuitului, pentru a detecta eventuale erori de cablare / lipse de contact ale componentelor cu placa,
- se verifică dacă circuitul de pe placă se comportă corect când e pus sub tensiune,
- se măsoară caracteristici ale componentelor individuale, dacă au puncte de test dedicate,
- se verifică dacă componentele programabile au fost încărcate corect cu software, și dacă software-ul trece corect de etapele sale de auto-testare,
- se verifică dacă circuitul se comportă corect în condiții excepționale (scurtcircuitări de componente, supratensiune, subtensiune, pierdere subită a tensiunii etc.)

2.1.2 Software

În cazul **sistemelor software**, nivelul de automatizare al testării este aproape întotdeauna maximul posibil, din simplul motiv că testarea automată este mult mai ieftină în software decât în hardware. În plus, în software avem avantajul de a putea efectua scenarii de test în mod repetat și foarte ușor. Rămân puține cazuri în care testarea manuală mai este necesară.

Atunci când un sistem software este dezvoltat, el este tratat asemănător cu un sistem hardware în dezvoltare, în sensul că testăm un "prototip" software, instanțiat de un calculator, care se vrea a fi reprezentabil pentru toate situațiile ulterioare în care produsul software va fi instanțiat și utilizat după distribuție și instalări.

Datorită naturii software-ului în general, "prototipul" instanțiat în timpul dezvoltării este practic identic cu fiecare instanțiere ulterioară a sistemului software după distribuție la utilizatori. Asta face ca, în cazul

sistemelor software, testarea ”în producție” (după livrare sau publicare) să fie un proces mult mai restrâns decât testarea în dezvoltare.

Un exemplu interesant de testare a instanțelor individuale de software este situația firmware-ului care este încărcat pe componente hardware programabile. Firmware-ul poate fi foarte complex, și pe deasupra poate avea cerințe de funcționare foarte stricte, care depind și de comonenta hardware. În aceste situații, fiecare componentă hardware va fi pusă într-un dispozitiv de testare care va rula scenarii de test dedicate pentru firmware-ul instalat pe acea componentă fizică. Acest proces se repetă pe fiecare componentă; chiar dacă firmware-ul este identic, instalarea lui poate să difere accidental între componentele fizice individuale, sau pot apărea defecte în timpul instalării. Posibilitatea acestor probleme impun necesitatea de a testa fiecare instalare de firmware pe fiecare sistem hardware.

Din același motiv sistemele software importante au și regimuri de funcționare ”de autotestare”, care pot fi invocate repetat în timpul funcționării. De exemplu:

- placa de bază a unui calculator personal conține firmware prevăzut cu regim de autotestare care, la fiecare pornire, rulează o serie de scenarii de test pentru a valida integritatea sistemului (software și hardware); practic, este testată fiecare instanțiere a firmware-ului (rularea), pentru fiecare placă de bază din fiecare calculator personal;
- kernelul Linux poate fi configurat (și în unele cazuri chiar este livrat preconfigurat) să testeze integritatea sistemului de fișiere la fiecare pornire, folosind `fsck`; în acest fel, kernelul testează sistemul de fișiere de fiecare dată când este instanțiat, pe fiecare dispozitiv pe care este configurat în acest fel.

Testarea software automată este o practică esențială în industrie, și va fi discutată în secțiunea următoare.

2.2 Testarea software automată

În dezvoltarea software, testarea automată se referă la practica de a scrie programe auxiliare programului principal, care conțin scenarii de test pentru programul principal. În limbaj uzual, această practică o numim simplu ”a scrie teste”.

Aceste teste sunt deseori scrise în același limbaj de programare în care este scris și codul-sursă al proiectului principal, dar nu este obligatoriu. Există chiar situații în care preferăm să scriem testele într-un *alt limbaj de programare* care le-ar ușura scrierea sau rularea. De exemplu, o aplicație scrisă în C++ poate avea teste de sistem scrise în Bash, sub formă de script-uri de shell.

Din punct de vedere al nivelului structural al componentei testate, distingem între *teste de unitate*, *teste de integrare* și *teste de sistem*.

Când scriem teste?

De la începutul dezvoltării. Testele sunt parte din proiectul dezvoltat și sunt începute odată cu codul-sursă al proiectului. Mai mult, vor fi adăugate teste pe măsură ce sunt adăugate componente, în pas cu dezvoltarea codului principal.

Abordarea *Test-Driven Development* (TDD) merge și mai departe: testele sunt scrise înainte de codul principal. De exemplu, testele unei componente sunt scrise înaintea componentei înseși, ceea ce oferă o linie utilă de urmărit în timpul dezvoltării și favorizează implementări concentrate pe specificații. Un dezavantaj este dificultatea de a menține disciplina necesară abordării TDD, care uneori nu oferă suficiente beneficii.

Autorul acestui document consideră că TDD oferă lecții bune de învățat și principii bune de urmărit, prioritizarea testelor înaintea codului principal fiind de multe ori un avantaj. Dar nu întotdeauna: TDD poate încuraja scrierea prea multor teste, înainte de a se ști cu oarecare certitudine toate comportamentele componentei (sau dacă componenta rămâne sau va fi eliminată). Se recomandă scrierea testelor înaintea codului, dar nu a tuturor testelor la care ne putem gândi. Ca întotdeauna, pragmatismul este important.

Dacă se întâmplă să fie adăugate componente fără să fie acoperite de suficiente teste, situația contribuie la fenomenul numit uzual *restanță tehnică* sau *datorie tehnică* (eng. technical debt). Există și alte forme de restanță tehnică, de exemplu o refactorizare întârziată sau o componentă importantă dar implementată incomplet. Restanțe tehnice apar în mod natural, dar trebuie recuperate de câte ori apare ocazia.

Când rulăm teste?

Cât de des posibil. În timpul dezvoltării unei componente se rulează testele ei după fiecare modificare, dar nu ale tuturor componentelor. Totodată, în timpul dezvoltării unei componente, i se adaugă teste noi. Implementarea este astfel validată în permanență de testări repetate.

Este recomandat ca testele să fie nu doar cuprinzătoare, dar și eficiente, pentru a reduce timpul lor de rulare și de a încuraja testarea după fiecare modificare.

Testele de unitate, fiind cele mai mici și mai simple, vor rula oricum foarte repede. De aceea se recomandă să fie cât mai multe și să fie rulate după fiecare modificare. Testele de integrare care implică o anume grupare de componente vor fi rulate când oricare din ele este modificată, dar după ce testele de unitate sunt rezolvate.

Deoarece testele de unitate și de integrare sunt, în mod normal, eficiente, ele sunt rulate "la grămadă", toate împreună, chiar dacă nu au fost modificate decât una sau două componente. Rularea *tuturor testelor* trebuie să fie *facilă*, și atunci dezvoltatorii le vor rula des. Desigur, cel mai des rulate teste rămân testele de unitate ale componentei în dezvoltare.

Testele de sistem se rulează mai rar, în funcție de specificul proiectului. Echipa de dezvoltare va stabili rutina de testare a întregului sistem.

2.2.1 Testele de unitate

Testele de unitate izolează unitățile individuale ale codului și conțin scenarii foarte specifice. Cu cât unitatea este mai simplă, cu atât va fi mai ușor să se scrie teste de unitate – un avantaj considerabil.

Deoarece un test de unitate se concentrează pe o singură unitate în izolare, este acceptabilă scrierea testului folosind informații detaliate despre implementarea unității. Cu alte cuvinte, unitatea este "deschisă" pentru testul de unitate. În cazul testelor de integrare această deschidere nu mai este neapărat valabilă, și este încurajată chiar "închiderea" unității față de test; un test de integrare nu ar trebui să folosească cunoștințe și detalii de implementare ale unei unități implicate – asta e sarcina testului de unitate. Desigur, acesta e un principiu al bunelor practici, nu o regulă, deci există teste de integrare care fac contrariul.

Este foarte normal ca o unitate să depindă de alte unități (sau chiar componente mai mari). În această situație, unitatea testată nu va putea fi inițializată corect decât dacă i se satisfac dependențele. Dar dacă pregătim o unitate U_2 pentru a fi oferită ca dependență în inițializarea unității U_1 , înseamnă că am creat un test care implică două unități, adică un *test de integrare*. Da, este un test foarte util și important, dar noi vrem un test de unitate, nu de integrare. Pentru a rezolva problema dependenței, va trebui să oferim lui U_1 o componentă care să suplinească U_2 , dar fără a imita comportamentul real al lui U_2 . Un astfel de suplinitor de componentă este folosit *numai și numai în testare*, și va trebui să imite U_2 suficient cât să îndeplinească dependența lui U_1 , dar **cu nimic mai mult**. Suplinitorul nu va fi folosit pentru nimic altceva, deci nu vom depune efort pentru a-l face complet funcțional. În niciun caz nu îl transformăm într-o componentă a sistemului.

În limbajul uzual, acești suplinitori se numesc *mock*, pentru că se prefac a fi componenta suplinită. Cuvântul "mock" are aici aceeași conotație pe care o are în numele personajului "Mock Turtle" (rom. "falsa țestoasă") din "Alice's Adventures in Wonderland", de Lewis Carroll, adică "imitație", "fals", "artificial", "prefăcut". Nu are înțelesul verbului "to mock" (rom. "a ridiculiza").

Ce este o unitate?

Alegerea *unității* în codul-sursă dezvoltat este la alegerea dezvoltatorului, dar în mod normal se urmăresc convenții în această decizie. Astfel, într-un limbaj OOP (Object-Oriented Programming) cum ar fi Java sau C#, unitățile sunt clasele individuale. Desigur, corespondența unitate - clasă este mai degrabă convențională, dar este foarte răspândită. Ca alternativă, am putea să avem un pachet ale cărui clase sunt atât de strâns interdependente încât nu pot fi izolate, și atunci pachetul este unitatea. Dar astfel de situații sunt excepționale, și convenția este încălcată doar cu justificare serioasă.

În limbaje procedurale, cum este C, unitățile sunt procedurile, iar testele de unitate vor fi concentrate pe câte o procedură. În limbajele funcționale, unitățile sunt funcțiile.

În limbajele multi-paradigmă vor fi mai multe posibilități pentru definiția unei unități. Proiectele de Rust, Python sau JavaScript / TypeScript pot avea, simultan, unități care sunt:

- *funcții pure*, fără efecte secundare, conform programării funcționale,
- *funcții cu efecte secundare*, conform programării procedurale sau OOP,
- *clase*, conform programării OOP cu clase,
- *obiecte-prototip*, conform programării OOP cu prototipuri.

A nu se confunda clasele din JavaScript și TypeScript cu clasele din Python, C++, Java sau C#. Chiar dacă toate aceste limbaje adoptă paradigma OOP, ele o fac în mod diferit:

- **Class-based programming** este stilul OOP adoptat de Python, C++, Java și C#;
- **Prototype-based programming** este stilul OOP adoptat de JavaScript și TypeScript.

Cu toate acestea, JavaScript și TypeScript oferă un artificiu sintactic care permite emularea claselor din celelalte limbaje folosind prototipuri. Cu alte cuvinte, `class` este un artificiu sintactic util în JavaScript și TypeScript, dar este de reținut că nu există de fapt clase propriu-zise în aceste două limbaje. Autorul acestui document recomandă insistent dezvoltatorilor JavaScript și TypeScript să înțeleagă foarte bine ce sunt prototipurile și cum funcționează, înainte să folosească artificul sintactic pentru clase, pentru a evita probleme viitoare.

După ce s-a stabilit care este unitatea de cod, putem discuta despre cum scriem fiecare scenariu de test, sub forma unui test de unitate.

Ce formă are un test de unitate?

În marea majoritate a cazurilor, testele de unitate sunt scrise în același limbaj de programare în care este scris proiectul principal.

În limbajele Java și C#, convenția este de a crea un proiect auxiliar în care vor fi scrise *clase-test*, fiecare clasă-test fiind corespunzătoare unei clase-unitate din proiectul principal. *Metodele clasei-test* sunt testele de unitate pentru clasa vizată, nu clasa-test, care de fapt le conține. O clasă-test conține și metode dedicate de pregătire și finalizare a fiecărui test individual, dacă este nevoie de astfel de pași. În mod convențional, clasa-test este numită *suită de teste*.

Un exemplu interesant este Python, pentru care există mai multe *framework-uri de testare*. Desigur, nu e singurul limbaj în această situație. Framework-ul de testare impune condiții pentru scrierea testelor de unitate, care trebuie cunoscute înainte de a decide care framework este folosit. De exemplu:

- framework-ul `PyUnit` seamănă cu framework-urile de test din Java și C#, adică testele de unitate sunt grupate într-o *clasă-test*, fiecare metodă a clasei-test fiind un test de unitate; sunt disponibile și metodele care pregătesc și finalizează un test individual; pentru un exemplu de test Python cu `PyUnit`, vezi 2.2.3;
- framework-ul `PyTest` oferă o altă abordare, mai degrabă procedurală: un test de unitate va fi orice funcție Python care este denumită cu prefixul `test_` (atenție la "underscore"!); iar framework-ul este invocat cu numele modulului Python în care sunt scrise astfel de funcții, grupându-le convenabil; pentru pașii de pregătire și finalizare, sunt disponibili *decoratori* cu care marcăm funcțiile de pregătire și finalizare.

O situație similară o întâlnim și în Rust, unde abordarea testării este bazată pe module care conțin funcții de test. Convenția în Rust este de a crea un modul numit `tests` la finalul unui fișier în care este scris codul care trebuie testat. În modulul `tests` scriem câte o funcție pentru fiecare test care vizează unități din fișierul curent. Pentru un exemplu de test Rust, vezi 2.2.2.

Python și Rust definesc oarecum diferit ce înseamnă un *modul*: în Python, un modul este un singur fișier, dar în Rust un fișier poate conține mai multe module (fiecare bloc `mod` dintr-un fișier este un modul individual). În mod implicit, fiecare fișier Rust este deja un modul, dar putem defini *submodule*, adică fiecare bloc `mod` dintr-un fișier Rust este un submodul al modulului-fișier.

În limbajele funcționale cum sunt Haskell sau PureScript, testele de unitate sunt funcții grupate pe module. Este păstrat conceptul de suită de teste, care sunt tot funcții.

La fel ca pentru Python, și pentru Haskell există mai multe framework-uri de testare, cu abordări diferite, de exemplu `HUnit`, care seamănă cu `JUnit` din Java și cu `PyUnit` din Python; sau `Hspec`, care seamănă cu `PyTest`.

De obicei, framework-urile de testare, indiferent de limbaj, nu fac diferențe între tipul de test implementat. Asta înseamnă că framework-urile de testare oferă facilități pentru a scrie orice fel de test, fiind răspunderea dezvoltatorului de a defini formele pe care le iau testele.

Așadar `JUnit`, `HUnit` și `PyUnit` sunt folosite pentru a scrie nu doar teste de unitate, ci orice fel de test, adică inclusiv teste de integrare și sistem, în ciuda denumirii. Acest fapt este la fel de valabil și pentru `PyTest` și `Hspec`.

*Este important de reținut că, în mod normal, un **test de unitate** urmează tiparul **setup-call-assert** indiferent de limbaj sau framework de testare. Acest tipar este structurat în următoarele etape:*

1. **setup**, adică testul începe efectuând pași de pregătire conform scenariului său, pregătind unitatea, valorile, obiectele necesare etc.; dacă testul este foarte simplu și nu necesită pregătiri, această etapă poate fi omisă;
2. **call**, adică testul apelează unitatea vizată, fie pentru a obține rezultate, fie pentru a-i schimba starea internă și a o interoga;
3. **assert**, adică testul evaluează rezultatele și valorile obținute în pasul **call**; dacă acestea se conformează așteptărilor, atunci testul este considerat trecut; dacă o evaluare este neconformă, testul este considerat picat.

Testele de integrare și testele de sistem urmează și ele acest tipar, în linii mari, dar îl extind sau îi repetă etapele în cadrul aceluiași test.

2.2.2 Exemplu simplu de test de unitate în Rust

În continuare este listat un exemplu de cod Rust cu un test de unitate.

Se observă că fișierul `add.rs` conține funcția publică `add()` care primește două numere întregi pe 64 de biți și întoarce tot un număr întreg pe 64 de biți.

După codul funcției urmează modulul de teste, adică `mod tests`, care va fi compilat doar atunci când dorim să testăm. Linia `#[cfg(test)]` este cea care informează compilatorul că modulul `tests` este compilabil doar când este activă configurația de testare a proiectului. Această configurație se activează automat când rulăm comanda `cargo test` într-un terminal în interiorul directorului care conține proiectul.

Listing 2.1: `add.rs`

```
1 pub fn add(left: u64, right: u64) -> u64 {
2     left + right
3 }
4
5 #[cfg(test)]
6 mod tests {
7     use super::*;
8
9     #[test]
10    fn it_works() {
11        println!("hello!"); // setup
12        let result = add(2, 2); // call
13        assert_eq!(result, 4); // assert
14    }
15 }
```

2.2.3 Exemplu simplu de teste de unitate în Python

În continuare este listat un exemplu de cod Python cu teste de unitate.

În stânga se află fișierul `calc.py` care conține funcția `calculate()`. Această funcție este *unitatea* pe care o testăm. Ea are trei parametri: `op`, `x` și `y`, adică operatorul și două valori pe care va fi aplicat operatorul. Funcția `calculate` este simplă: primește operatorul și două valori, și returnează rezultatul lor. Se observă faptul că operatorul poate fi `'+'`, `'-'`, `'*'`, `'/'`, `'power'` sau `'log'`.

În dreapta se află fișierul `test_calc.py` care conține mai multe funcții, fiecare dintre acestea fiind un *test de unitate*, adică un test pentru funcția `calculate()`. Se observă că fiecare funcție de test este denumită corespunzător, conform convenției `pytest`, amintit mai sus. Adică numele fiecărei funcție de test are prefixul `test_` urmat de o foarte scurtă descriere a scenariului testat.

Listing 2.2: `calc.py`

```
1 import math
2
3 def calculate(op, x, y):
4     if op == '+':
5         return x + y
6     elif op == '-':
7         return x - y
8     elif op == '*':
9         return x * y
10    elif op == '/':
11        return x / y
12    else:
13        return calculate_exp_log(op, x, y)
14
15 def calculate_exp_log(op, x, y):
16     if op == 'power':
17         return x**y
18     if op == 'log':
19         return math.log(x, y)
```

Listing 2.3: `test_calc.py`

```
1 import calc
2
3 def test_calculator_div0():
4     # setup
5     zero_div_exception = False
6
7     # call
8     try:
9         n = calc.calculate('/', 1, 0)
10    except ZeroDivisionError:
11        zero_div_exception = True
12
13    # assert
14    assert zero_div_exception
15
16 def test_calculator_add():
17     # call
18     n = calc.calculate('+', 1, 1)
19
20    # assert
21    assert n == 2
22
23 def test_calculator_sub():
24     n = calc.calculate('-', 1, 1)
25     assert n == 0
26
27 def test_calculator_multiply():
28     n = calc.calculate('*', 1, 1)
29     assert n == 1
30
31 def test_calculator_div():
32     n = calc.calculate('/', 1, 1)
33     assert n == 1
34
35 def test_calculator_power():
36     n = calc.calculate('power', 2, 4)
37     assert n == 16
38
39 def test_calculator_log2():
40     n = calc.calculate('log', 16, 2)
41     assert n == 4
```

2.2.4 Testele de integrare

Când un test vizează o singură unitate, el este *test de unitate*. Atunci când un test vizează două sau mai multe unități care interacționează, atunci el este un **test de integrare**. Astfel, spectrul testelor trece relativ abrupt de la *test de unitate* la *test de integrare*.

Din nou, trebuie reținut că cele mai folosite framework-uri de testare, indiferent de limbaj, *nu fac distincția*

între teste de unitate și integrare. Pentru framework-uri, toate sunt *teste automate*. Distincția apare însă în modul în care testele sunt scrise și câte unități sunt implicate în scenariul testat.

În ciuda acestei distincții, testele de integrare păstrează în mod foarte vizibil tiparul **setup-call-assert**, același ca la testele de unitate. În mod nesurprinzător, etapa **setup** al unui test de integrare este mai consistentă, deoarece trebuie să inițializeze mai mult de o unitate.

Spre deosebire de testele de unitate, se poate spune că testele de integrare se concentrează mai mult pe *contextul* în care două sau mai multe unități trebuie să funcționeze, adică fiecare unitate este contextul pentru fiecare cealaltă unitate din cadrul testului.

Dar asemănător cu testele de unitate, testele de integrare rămân conștiente de detaliile de implementare ale fiecărei unități, chiar dacă nu se vor concentra pe acele detalii decât dacă o cere scenariul de test.

2.2.5 Testele de sistem

Un **test de sistem** ia întregul sistem, cu toate unitățile / componentele lui, și îl trece prin situații care se apropie de utilizarea reală, de scopul căruia îi este destinat. Este responsabilitatea testului de sistem să simuleze *mediul* în care sistemul va fi pus, să ofere intrări și să evalueze conformitatea ieșirilor cu așteptările.

Datorită faptului că testele de sistem variază prea mult între tipuri de sistem, ele nu vor fi discutate în detaliu în acest document.

Chapter 3

Testarea bazată pe modele

Testarea bazată pe modele (eng. Model-Based Testing, MBT) este o practică specifică *testării automate* (dar nu exclusivă).

În foarte multe din cazuri, comportamentul sistemului poate fi modelat teoretic, chiar și cu modele simple. Putem folosi această posibilitate pentru a *genera teste* ale sistemului. Asta înseamnă:

1. crearea unui model teoretic pentru un aspect al comportamentului sistemului,
2. crearea unui generator de scenarii de test, pe baza modelului,
3. crearea unui executor de astfel de scenarii de test.

Sintagma **aspect al comportamentului sistemului** se referă la faptul că *nu ne concentrăm pe întregul sistem*, ci doar pe un aspect. Motivul pentru care nu luăm în considerare întregul sistem este faptul că modelul rezultat ar fi prea mare și prea complex pentru a fi ușor de gestionat în testare. Astfel, recurgem la izolarea unor aspecte ale sistemului, de interes prioritar, pentru a putea crea un model mai simplu.

E important de reținut că se pot crea *mai multe modele* pentru același sistem, fiecare concentrat pe câte un aspect (sau mai multe deodată) ale sistemului. Tot ce ne interesează este să creăm câte un model *simplu* și *clar* pentru câte un aspect relevant al sistemului. Dar ce este un model?

Modelul teoretic poate fi orice construcție teoretică, dacă este suficient de riguroasă. Exemplul clasic, des folosit și foarte util în multe situații este automatul cu stări finite. Dacă sistemul testat este un dispozitiv electronic simplu, deja avem două stări cu tranziții între ele: starea *pornit* și starea *oprit*.

Generatorul de scenarii de test este un pas intermediar, opțional, între model și execuția testului. Este necesar dacă modelul poate produce *în masă seturi de parametri* pentru scenariile de test, adică le poate enumera într-o listă lungă. În acest caz, generatorul de scenarii de test va trece peste fiecare set de parametri, și va produce un scenariu de test, pe care ori îl va trimite imediat executorului de teste, ori îl va salva pentru execuție ulterioară. Momentul în care sunt executate scenariile de test va fi ales de dezvoltator, în funcție de necesități.

Dacă modelul nu este implementat într-un fel în care să enumere automat seturi de parametri, atunci abordarea este de a integra modelul gata implementat direct în testele automate. Această abordare poate părea simplă sau ușoară la început, dar se dovedește a fi mai puțin sustenabilă decât o implementare a modelului care să poată enumera seturi de parametri, în mod decuplat de execuția scenariilor.

Setul de parametri pentru un scenariu de test este o colecție de valori care reprezintă, în mod abstract, scenariul prin care sistemul trebuie trecut, cu intrările oferite și cu ieșirile așteptate. Setul de parametri va fi primit de către executorul de scenarii de test, și îl va trata ca pe instrucțiuni de testare. Urmărind exemplul de mai sus, al modelului de tip automat cu stări finite, un astfel de set de parametri ar fi:

- starea sistemului la începutul testului,
- tranziția de efectuat,
- valori de oferit sistemului ca intrare,
- starea sistemului după tranziție.

Acest set de parametri va fi interpretat de executorul de scenarii ca pe o "rețetă" foarte compactă și abstractă a situației în care trebuie pus sistemul.

De exemplu, automatul cu stări finite generează setul `(4, load, [42, 19, "gxa"], 9)`. Acest set de parametri reprezintă scenariul în care executorul de test va inițializa sistemul în starea `4`, îi va da comanda `load` (reprezentată ca tranziție de stare), oferind ca intrare valorile `[42, 19, "gxa"]`. La finalul comenzii, sistemul va trebui să se regăsească în starea `9`. Dacă automatul a fost implementat corespunzător, atunci va putea genera sute sau mii de astfel de seturi parametri *înaintea execuției oricărui test*.

Executorul de scenarii efectuează testele propriu-zise, consumând câte un set de parametri. Executorul este cel care se asigură că sistemul este pornit în starea cerută de scenariu (în setul de parametri), se asigură că sistemul primește corect comanda și datele intrare cerute de scenariu, și, la final, verifică dacă sistemul este în starea așteptată de scenariu. Această verificare, fiind aplicată pe sistemul deja implementat, este de fapt *un test de integrare sau de sistem*, adică are ca rezultat un verdict: *trecut* sau *picat*.

Testele generate de modele sunt de regulă teste de integrare și sistem, unde sunt foarte valoroase din cauză că pot explora spațiul stărilor sistemului și al configurațiilor sale. Dar se întâmplă să fie folosite și ca teste de unitate, dacă unitatea acceptă un spațiu mai mare de intrări, cu mai multe combinații relevante de testat.

Dar nu vom depune efortul de a scrie un generator cu model pentru funcția `calc.calculate()` din Secțiunea 2.2.3, chiar dacă acceptă, ca intrări, toate numerele care pot fi exprimate în Python. Nu toate combinațiile de numere sunt relevante. Exemplul din Secțiunea 2.2.3 s-a concentrat pe testarea operațiilor obișnuite, cu câte un singur set de valori, cu excepția împărțirii, care prezintă singularitatea împărțirii cu zero.

În orice caz, categoria de test (unitate, integrare, sistem) este o grijă secundară atunci când implementăm testare bazată pe modele. Grijă principală este crearea unui model, a generatorului de scenarii și a executorului de scenarii. Adică testarea bazată pe modele. Existența testării bazate pe modele într-un proiect este întotdeauna un semn de competență al dezvoltatorilor. Fiabilitatea sistemului nu are decât de câștigat, mult.

Exemplul dat aici este, desigur, doar ilustrativ și foarte simplu. Pot exista modele foarte complexe acolo unde problema le cere. Este o practică bună în dezvoltarea testelor să ne întrebăm încă de la început: "Pot să modelez componenta cumva, să nu mai scriu teste, ci să le generez automat din model?". Desigur, răspunsul nu este întotdeauna afirmativ. Dar este suficient de des afirmativ, și în aceste cazuri, abordarea testării bazate pe modele este o idee excelentă. Alternativa este fie de a scrie manual sute sau mii de teste, fie de a scrie manual doar câteva teste, dar de a omite astfel scenarii care sunt relevante și importante.

Chapter 4

Invarianții

Testarea discutată în Capitolul 2 este un aspect al **dezvoltării** și **producției**. După încheierea dezvoltării și producției intervin alte metode de validare a sistemului. Un foarte bun exemplu de astfel de validare o constituie **invarianții**. Așadar, se recomandă a nu se confunda testarea cu implementarea invarianților.

Este important de reținut că termenul invariant are mai multe înțelesuri în mai multe domenii. Aceste înțelesuri nu sunt neapărat compatibile. În matematică, invarianța este mai degrabă o proprietate a unor obiecte de a rămâne neschimbate în urma unor operații sau transformări. În fizică, termenul invarianță este întâlnit cu un înțeles asemănător, dar aplicat pe descrieri teoretice ale sistemelor fizice (cum ar fi covarianța Lorenz din teoria restrânsă a relativității). Este întâlnit și termenul de simetrie pentru invarianți.

În acest document, folosim termenul invariant cu înțelesul de condiție impusă sistemului și verificată de sistem; acest înțeles este întâlnit mai des în inginerie. Nu este un înțeles la fel de riguros ca cel din matematică, dar este extrem de util, mai ales în fiabilitate și în teoria sistemelor tolerante la defecte, care trebuie să se poată redresa în cazul erorilor.

În schimb, termenul invariant este foarte bine definit în știința și ingineria calculatoarelor, cum va fi discutat în continuare.

4.1 Invarianți în știința și ingineria calculatoarelor

Descris în cel mai simplu mod, **un invariant** este o **condiție** pe care sistemul însuși o verifică, fiind astfel proiectat și implementat. Desigur, sistemul este implementat și să reacționeze la cazurile în care invariantul este încălcat, chiar dacă reacția sistemului este doar de a aprinde un LED de avertizare. În alte cazuri, încălcarea unui anume invariant va determina sistemul să reacționeze "de urgență", de exemplu întreruperea unui circuit electric sau anularea unei lansări de rachetă.

Cel mai des întâlnit invariant apare în buclele din limbajele de programare imperative și procedurale. Astfel, orice buclă **for** sau **while** este scrisă cu o condiție care *rămâne adevărată în interiorul unei iterații*. O putem numi **invariant de iterație** sau **invariant de buclă** (eng. loop invariant).

În interiorul unei singure iterații, invariantul buclei (condiția sa) este garantat adevărat. La începutul următoarei iterații invariantul este evaluat din nou, iar în cazul în care este încălcat, bucla este oprită. La fel se întâmplă și în cazul în care codul buclei schimbă în mod intenționat valorile de care depinde condiția.

4.2 Exemple, în general

Un exemplu comun de sistem prevăzut cu invarianți este instalația electrică dintr-o locuință. Când consumatorii din locuință nu suprasolicite instalația, totul funcționează normal. Dar, în caz excepțional, unul din consumatori se defectează și introduce un scurtcircuit. Curentul din instalație crește repede (*foarte repede*) și circuitul trebuie întrerupt imediat pentru a preveni defectarea celorlalți consumatori, și pentru a preveni supraîncălzirea conductorilor, situație care poate să producă ușor incendii. În această situație, instalația însăși conține un mai mulți disjunctori ("siguranțe") care, prin construcția lor, impun o limită de intensitate în

circuit ca *invariant*. În funcție de tipul unui disjunct, acesta va impune limite de intensitate în diferite situații:

- un disjunct magnetotermic impune o limită de 6 A, 8 A, 10 A etc. pentru intensitatea curentului care îi parcurge bornele,
- un disjunct diferențial impune o limită de câteva zeci de miliamperi a diferenței dintre intensitățile curenților care parcurg cele două perechi de borne ale sale, păstrând astfel echilibru între curentul de "intrare" în instalație și cel de "ieșire" din instalație.

Exemplul unui disjunct care impune o limită de intensitate a curentului este, desigur, un exemplu de *implementare fizică* a unui invariant. În acest exemplu sistemul este, desigur, întreaga instalație. De asemenea, a nu se considera că disjunctul este invariantul: el *impune un invariant*, anume limita de intensitate. **Invariantul este condiția impusă**, adică o noțiune abstractă, nu implementarea ei în hardware sau software.

Nu doar că un invariant este o noțiune abstractă, dar este, de foarte multe ori, **provenită din teoria** relevantă sistemului. Condiția este, astfel, exprimată în termenii teoriei. În cazul disjunctelor, invariantii sunt proveniți din electrotehnică, unde discutăm despre circuit și intensități. Chiar dacă e vorba de electrotehnică, o ramură a fizicii, acest exemplu nu este un invariant în sens matematic, ci mai degrabă în sens ingineresc.

Un alt exemplu găsim în contabilitate. Un sistem informatic pentru gestiunea contabilității va verifica, regulat, dacă este respectat principiul de contabilitate în partidă dublă. În această situație, sistemul va ține evidența conturilor proprii organizației, dar și a conturilor externe. În acest fel, orice transfer către terți va apărea ca o debitare dintr-un cont propriu și o creditare a unui cont extern. Asta din cauză că sistemul ține evidența conturilor externe cu care interacționează. Acesta este un alt exemplu de **invariant provenit din teorie**.